

Evidence Visibility Shapes Risk Behavior in a Controlled LLM Patch-Verifier Study

Anonymous Authors
Anonymous Institution

Abstract—Patch-verification decisions made by large language models (LLMs) are meaningful only relative to the evidence visible at review time. Existing evaluations often report whether an LLM accepts or rejects a patch without making this evidence boundary explicit, which makes it difficult to separate model capability from visible-test evidence, tool-summary anchoring, or prompt-induced caution. This paper introduces the Evidence-Visibility Protocol (EVP-8), a hidden-evaluator protocol for candidate patch verification. EVP-8 reviews 98 candidate patches across seven cumulative evidence levels while withholding evaluator-only correctness labels until post-decision analysis. In three repaired v0.3 runs, Qwen reached 95.24% E6 correct recall with 83.33% accepted precision, DeepSeek reached 80.95% E6 correct recall with 80.95% accepted precision, and Gemini reached 95.24% E6 correct recall with 80.00% accepted precision. The three models still accepted 4-5 of 77 non-correct candidates at E6. E6 ablations and contestation tests show that prompt framing can shift decisions toward tool-following, abstention, or risk routing; a supplemental 31-case hard-negative stress matrix confirms that safer handling can come through escalation rather than strict rejection. These results support a bounded software-engineering claim: EVP-8 reveals when LLM patch verifiers behave as tool followers, abstainers, or risk routers under explicit evidence boundaries, not whether they provide general autonomous correctness verification.

I. INTRODUCTION

Automated patch generation and LLM-assisted review are increasingly positioned near software merge decisions. This creates a practical software-quality question: when a candidate patch appears plausible, which visible evidence is sufficient for a verifier to accept it rather than reject it or escalate it for human review? Prior automated program repair work has shown that plausible or test-passing patches can still be incorrect [1], [2], so a merge decision cannot be reduced to surface plausibility alone.

The difficulty is not only whether an LLM can read code. It is that a verifier’s decision may change when issue context, patch structure, static status, visible tests, regression checks, diagnostics, or tool summaries become visible. If these evidence fields are not controlled, an evaluation may conflate model judgment with evidence presentation. In particular, authoritative-looking tool summaries can encourage acceptance even when the underlying semantic correctness remains unknown.

This paper studies candidate patch verification as an evidence-conditioned merge-gate task. A verifier receives a candidate patch and a predefined model-visible evidence packet, then emits one of three decisions: accept, reject, or escalate. Correctness labels and hidden evaluator outcomes are

joined only after the decision. This design follows the intuition behind reject-option and selective-classification settings [3], [4], [5], but applies it to software patch verification. The current empirical scope is intentionally controlled: the paper-facing main result is a three-model repaired v0.3 analysis for Qwen, DeepSeek, and Gemini, supported by E6 ablations and tool-contestation evidence, rather than a broad claim about all LLM verifiers.

The paper makes three contributions:

- It defines EVP-8, a hidden-evaluator evidence-visibility protocol for measuring accept, reject, and escalation behavior in candidate patch verification.
- It reports repaired Qwen, DeepSeek, and Gemini v0.3 label-conditioned results showing that visible executable and tool evidence can unlock correct-patch acceptance while introducing bounded false-accept risk.
- It analyzes E6 rule-only, no-verdict, tool-contestation, coverage-contestation, and hard-negative stress-test conditions to separate deterministic tool evidence, verdict-like anchoring, safe handling, strict correction, and prompt-induced conservatism.

The claim is deliberately bounded. The results do not establish reliable autonomous patch correctness verification, nor do they show that LLM decisions consistently outperform deterministic baselines. They show that evidence visibility is a measurable experimental variable that should be controlled and reported in LLM patch-verifier studies.

II. BACKGROUND AND MOTIVATION

Automated program repair research has long distinguished plausible patches from correct patches. Generate-and-validate and semantic-repair systems make this distinction visible, while controlled benchmarks expose why available tests are not complete correctness oracles [1], [6], [7], [8], [9], [10], [11], [12], [13]. EVP-8 studies the downstream verifier setting: after a candidate patch exists, the question is not how it was generated, but whether a merge-gate decision is justified by the evidence visible to the verifier.

Software testing research also motivates the hidden-evaluator design. The oracle problem means that deciding correctness is itself a validity boundary [14]. EVP-8 therefore keeps evaluator-only labels separate from model-visible evidence and joins them only after decisions have been produced.

LLM-based repair, code generation, and code-editing studies show that LLMs can produce or inspect patches [15], [16],

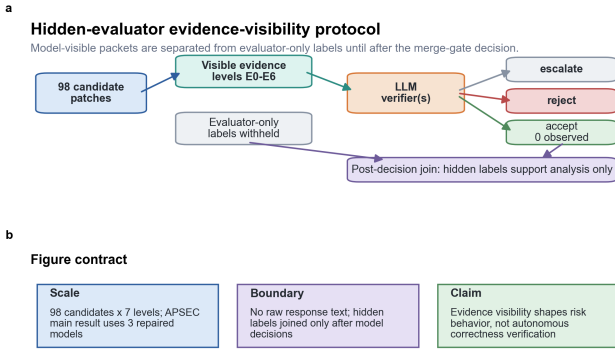


Fig. 1. Hidden-evaluator evidence-visibility protocol.

[17], [18]. However, verifier behavior is not identical to generation performance. A patch reviewer must decide whether visible evidence is enough to accept, reject, or escalate. This makes the task closer to code review and risk triage than to benchmark success alone [19], [20].

Recent software-agent benchmarks further show why task framing and evaluation protocol matter for code-oriented LLM systems [21], [22]. Finally, LLM-as-judge and automation-reliance work warn that model judgments and automation outputs require controlled protocols and explicit boundaries [23], [24], [25]. The protocol in this paper responds to that concern by making evidence visibility an explicit variable rather than an implicit property of the prompt.

III. EVIDENCE-VISIBILITY PROTOCOL

EVP-8 evaluates a candidate patch under a fixed evidence packet. The model-visible packet contains only the evidence level assigned to that condition. The evaluator-only layer contains hidden correctness labels and oracle outcomes used only after the verifier emits its decision.

The reviewed unit is a candidate patch. The decision space is accept, reject, or escalate. Escalation is treated as a human-review routing decision, not as semantic rejection. This distinction is important because safe handling of a risky patch can occur through escalation even when strict correction does not occur.

EVP-8 uses seven cumulative evidence levels:

The current packet set contains 98 candidate patches from 6 projects and 21 BugsInPy tasks [11]. It contains 21 correct patches and 77 non-correct patches under hidden F2P/P2P-broad evaluator labels. The candidate mix is intentionally skewed toward negative and partial cases because the merge-gate risk is false acceptance rather than classification accuracy on a balanced benchmark.

The 76 issue-not-fixed negatives plus one regression negative form the 77 non-correct candidates. Project counts are PySnooper 10, cookiecutter 19, httpie 6, thefuck 4, tqdm 7, and youtube-dl 52.

Five selected models produced complete parse-valid decisions on an earlier frozen E0-E6 packet set, but the paper-facing main result now uses repaired Qwen, DeepSeek, and

Gemini v0.3 label-conditioned analyses plus the E6 ablation package. The earlier five-model aggregate synthesis is used only descriptively and not as evidence of final model superiority. The repaired three-model main table removes the previous single-/two-model visibility weakness, but it still does not prove broad LLM superiority over deterministic tool summaries.

IV. EXPERIMENTAL DESIGN

The experiment asks three research questions. RQ1 asks whether repaired accept-aware evidence changes Qwen, DeepSeek, and Gemini label-conditioned decisions across E0-E6. RQ2 asks whether verdict-like deterministic tool summaries anchor E6 decisions. RQ3 asks: when visible evidence is challenged, do models correct wrong accepts or route them to escalation? Tool-contestation, current-98 coverage-contestation, and the hard-negative stress matrix are three evidence sources for RQ3 rather than separate main questions. The hard-negative stress matrix remains a supplemental stress test because its third project comes from curated no-API stress-source variants, not a pure agent-generated realistic cohort.

All metrics are computed after post-decision hidden-label join. The main metrics are accepted precision, correct recall, false accept rate, false reject rate, and escalation rate. Accepted precision measures the correctness of accepted patches. Correct recall measures how many correct patches were accepted. False accept rate measures how often non-correct candidates were accepted. Escalation rate measures routing to human review.

The baseline boundary is explicit. Always-escalate, always-reject, and always-accept are deterministic reference policies calculated from label totals. Uniform random three-way is an expected reference policy, not a stochastic experiment. The completed deterministic baseline is rule-only visible-tool. Qwen E0 is an observed model condition, not a deterministic no-tool verifier. Majority-vote and a separate E0/no-tool deterministic verifier are not reported as completed because current tracked summaries do not contain candidate-level aligned decision records.

A. Implementation and artifact boundary

All repaired main runs used the same frozen prompt template, `evp8_visible_evidence_merge_gate_v0_2`, the same E0-E6 packet construction, temperature 0.0, and a 4096-token output cap. The three repaired full runs each produced 686 parse-valid records, covering 98 candidates across seven levels. Parse validity required a schema-conforming accept, reject, or escalate decision with required structured fields; invalid JSON or missing decisions were execution-chain failures, not model decisions. Cost telemetry was tracked when available, but cost comparison is not used as a paper-facing claim. The paper-facing artifacts are raw-output-free summaries, scripts, generated packets, audits, and anonymized aggregate tables; raw provider responses, rendered prompts, patch diffs, local configs, and credentials remain excluded from tracked outputs.

TABLE I
CUMULATIVE EVP-8 EVIDENCE LEVELS.

level	visible evidence	meaning
E0	Issue summary and candidate patch diff	minimal review context
E1	Structured changed-file/function map	patch surface information
E2	Patch application and static-check status	basic feasibility evidence
E3	Visible fail-to-pass test results	issue-relevant executable evidence
E4	Visible pass-to-pass regression results	regression-safety evidence
E5	Additional diagnostics	broader tool evidence
E6	Deterministic merge-gate summary	summarized tool verdict

TABLE II
EVP-8 CANDIDATE COMPOSITION.

candidate type	count	hidden label	purpose
Correct reference	21	correct	correct-patch recall
Buggy no-op	21	Issue not fixed	obvious negative
Irrelevant patch	14	Issue not fixed	plausibility trap
Partial fix	41	Issue not fixed	semantic incompleteness
Regression patch	1	regression	regression risk

All paper-facing claims are constrained by a setting-validity audit. The audit checks run coverage, parse validity, raw-output-free summaries, post-execution label joins, prompt-boundary conditions, baseline feasibility, and exclusion of the earlier invalid setting. The audit passed only for bounded claims.

V. RESULTS

A. Visible executable evidence changed three repaired model policies

In the repaired Qwen, DeepSeek, and Gemini v0.3 runs, E0-E2 produced no accepted correct patches for Qwen and DeepSeek and only a single incorrect Gemini accept at E1. Once visible executable evidence entered the packet, all three models began accepting correct patches, but their risk policies diverged. Qwen reached 80.95% correct recall at E3 and 95.24% at E6. DeepSeek reached 61.90% correct recall at E3 and 80.95% at E6. Gemini reached 95.24% correct recall from E3 through E6, but accepted five of 77 non-correct candidates at E6. The repaired three-model table therefore strengthens the evidence-visibility finding while preserving the false-accept risk boundary.

The three-model result supports an evidence-visibility claim, not a monotonic correctness claim. More evidence changed policy behavior and unlocked acceptance, but intermediate-level non-monotonic behavior in the full artifact and the shared partial/regression false accepts show that visible evidence did not become a correctness oracle.

B. Verdict-like evidence affected policy behavior

The E6 ablation is a separate verdict-field ablation package rather than the repaired v0.3 E0-E6 main table. It compares the deterministic rule-only visible-tool baseline, full E6 model conditions, and E6-no-verdict conditions to test whether model decisions add value beyond following visible tool verdict fields. Therefore, the DeepSeek E6-full row below should be read as ablation evidence, while the repaired v0.3 DeepSeek

E6 row in Section 5.1 is the main E0-E6 evidence-visibility result.

Qwen E6-full and rule-only produced similar correct recall, accepted precision, and false accept rates. Qwen E6-no-verdict remained close to Qwen E6-full. DeepSeek E6-no-verdict removed false accepts in this cohort, but correct recall dropped to 52.38% and escalation increased to 14.29%. The safer behavior therefore appears to be partly abstention-driven rather than strict semantic discrimination.

This is an important negative result for the LLM-verifier claim. The deterministic rule-only baseline was already strong: it reached 95.24% correct recall and 80.00% accepted precision, compared with 95.24% and 83.33% for Qwen E6-full. The goal of EVP-8 is not to show that LLMs dominate rule-only tool summaries, but to expose when LLM decisions collapse into tool-following, abstention, or prompt-induced conservatism. The current evidence therefore does not justify claiming a large LLM gain over the tool summary. The value of the LLM conditions in this draft is narrower: they expose how model policy changes when verdict-like fields are removed or challenged, and they show whether risky accepts are routed to escalation rather than autonomous acceptance.

The Wilson intervals are wide, so the ablation should be read as bounded risk-policy evidence rather than as a ranking of model quality. The full CI table remains part of the analysis package rather than a main-body table.

C. Tool-contestation shifted risky accepts to escalation, not strict correction

On the EVP-8-HARD cohort, tool-contestation evaluated 47 candidates for both Qwen and DeepSeek. For the known tool false-accept opportunity set, DeepSeek shifted 9 tool false accepts to 9 escalations and 0 strict rejects. Qwen shifted 9 tool false accepts to 8 escalations, with 0 strict rejects and 1 repeated accept.

This result supports safe handling through escalation. It does not support a claim that tool-contestation reliably identifies

TABLE III
THREE-MODEL MAIN RESULTS AT SELECTED EVIDENCE LEVELS.

model	level	accept	correct accept	false accept	accepted precision	correct recall	false accept rate	escalation rate
Qwen	E0	0	0	0	n/a	0.00%	0.00%	75.51%
Qwen	E3	20	17	3	85.00%	80.95%	3.90%	4.08%
Qwen	E6	24	20	4	83.33%	95.24%	5.19%	0.00%
DeepSeek	E0	0	0	0	n/a	0.00%	0.00%	96.94%
DeepSeek	E3	16	13	3	81.25%	61.90%	3.90%	8.16%
DeepSeek	E6	21	17	4	80.95%	80.95%	5.19%	4.08%
Gemini	E0	0	0	0	n/a	0.00%	0.00%	98.98%
Gemini	E3	25	20	5	80.00%	95.24%	6.49%	3.06%
Gemini	E6	25	20	5	80.00%	95.24%	6.49%	0.00%

Three-model repaired evidence metrics

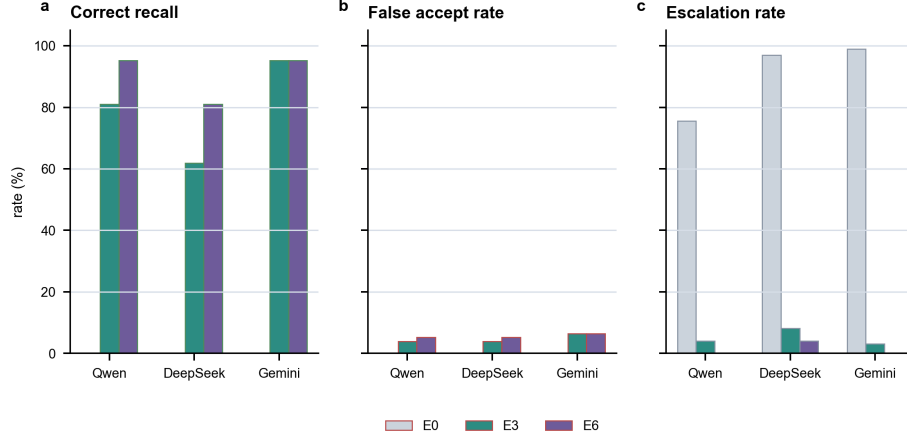


Fig. 2. Three-model repaired evidence metrics.

TABLE IV
E6 DETERMINISTIC AND NO-VERDICT ABLATION.

condition	accept	reject	escalate	accepted precision	correct recall	false accept rate	escalation rate
Rule-only	25	73	0	80.00%	95.24%	6.49%	0.00%
visible-tool							
DeepSeek E6-full	23	75	0	82.61%	90.48%	5.19%	0.00%
DeepSeek E6-no-verdict	11	73	14	100.00%	52.38%	0.00%	14.29%
Qwen E6-full	24	74	0	83.33%	95.24%	5.19%	0.00%
Qwen E6-no-verdict	23	74	1	82.61%	90.48%	5.19%	1.02%

semantic incorrectness, because strict correction remained zero for both models.

The opportunity-set Wilson intervals are wide and are retained in the analysis package rather than expanded into a separate main-body table.

D. Coverage-contestation removed repeated false accepts by becoming highly conservative

The current-98 coverage-contestation condition tested whether a stronger prompt could challenge visible-test-only acceptance without changing the frozen E6/no-verdict packet set. It removed repeated false accepts for Qwen, DeepSeek, and Gemini, reducing the false accept rate on 77 non-correct candidates to 0.00% for all three models. This is prompt-sensitivity evidence, not a new main result, because the same condition also collapsed correct-patch acceptance. DeepSeek

and Gemini accepted no correct patches, and Qwen accepted only 2 of 21 correct patches.

The result answers a narrow prompt-setting question. A model can be instructed to challenge coverage sufficiency and avoid visible-test-only acceptance on this frozen cohort, but the observed mechanism is mostly conservative triage rather than semantic discrimination. Therefore the paper should not claim that coverage-contestation improves autonomous verification. The supported claim is that prompt framing can move false-accept risk into reject/escalate outcomes while imposing a large correct-recall cost.

E. A hard-negative stress matrix reduced false accepts through escalation

After the current-98 prompt-sensitivity result, we constructed a separated hard-negative stress cohort to test visible-

pass/hidden-fail cases more directly. The cohort contains 31 hidden-failing candidates across PySnooper, cookiecutter, and scrapy. The visible-tool baseline accepted all 31 cases. Because the scrapy cases are curated no-API stress-source partial variants, this is a hard-negative stress-test supplement rather than a pure agent-generated realistic cohort.

Across Qwen, DeepSeek, and Gemini, the current merge-gate prompt produced 62 repeated false accepts in 93 model-condition records. The coverage-contestation prompt reduced repeated false accepts to 12/93. The reduction came entirely through escalation: strict rejects remained 0 in both conditions.

This stress result strengthens the risk-triage interpretation. It is bounded triage evidence: the stronger prompt can route many visible-pass/hidden-fail candidates away from autonomous acceptance, including all Gemini stress cases and all DeepSeek stress cases. It does not show semantic correction, because no condition strictly rejected the hard negatives. Correct recall is also undefined in this all-negative cohort.

F. E6 false accepts were concentrated in partial and regression negatives

The most important failure mode is not the average E6 score but the remaining E6 false accepts among 77 non-correct candidates. Aggregate false-accept anatomy shows that Qwen and DeepSeek each accepted three partial fixes and one regression patch, while Gemini accepted four partial fixes and one regression patch. In the single regression negative included in EVP-8, all three repaired models accepted it at E6; this is a severe failure signal, not a statistical claim about regression-safety failures. The partial-fix rows show a broader semantic-incompleteness pattern concentrated in youtube-dl tasks.

The sanitized case-level export records candidate id, project, task, negative type, E6 decision, no-verdict decision where available, and compressed rationale categories, but excludes raw response text, full rationale text, rendered prompts, patch diffs, and credentials. The table supports failure anatomy, not a claim that the complete model rationale has been audited semantically.

VI. DISCUSSION

The main implication is that LLM patch verification should be evaluated with explicit evidence boundaries. The same candidate patch may be treated differently when visible tests, regression checks, diagnostics, or deterministic tool summaries are introduced. Reporting only an aggregate accept/reject rate would hide this dependence.

Prompt sensitivity is part of the measurement target rather than only a nuisance variable. EVP-8 exposes whether a prompt reduces false-accept risk through semantic discrimination or through abstention-driven routing. In the current evidence, the safer coverage-contestation behavior was mostly abstention and escalation, not strict correction.

The results also clarify the role of escalation. Escalation can be useful for software-quality workflows because it routes risky cases away from autonomous acceptance. However, escalation is not strict correction, and aggressive contestation

can destroy correct-patch recall. This distinction matters for deployment: a verifier that escalates risky candidates may reduce unsafe automation, but it has not proven semantic incorrectness or preserved useful acceptance.

The strongest current contribution is methodological. EVP-8 provides a reproducible way to separate model-visible evidence from hidden evaluator labels, report accept/reject/escalate outcomes, and connect each claim to a validity gate. This is why the paper is framed as a controlled protocol and measurement study rather than as a new repair or verification algorithm.

Several reviewer concerns remain bounded rather than eliminated. The cohort is small, Wilson intervals are wide, repaired Qwen, DeepSeek, and Gemini are the main model conditions, majority-vote cannot be computed from the current tracked summaries, and the hard-negative stress matrix is a stress-test supplement rather than a pure realistic agent-patch cohort. Most importantly, the current paper-facing main result is three-model but still not broad-model. These are not hidden weaknesses; they are the boundary conditions under which the current claims are valid.

VII. THREATS TO VALIDITY

Internal validity. Prompt wording, output schema, and evidence formatting may influence decisions. The study mitigates this risk through frozen packets, tracked prompt-boundary checks, parse-validity audits, raw-output-free summaries, and post-decision label joins. These controls reduce setup risk but do not make the results independent of the evaluated protocol versions.

Construct validity. The accept/reject/escalate decision space simplifies real review workflows. Escalation is a routing outcome, not proof of semantic rejection. Hidden evaluator labels are used only for post-decision analysis and should not be interpreted as model-visible truth.

External validity. The main evidence comes from a 98-candidate EVP-8 cohort, an EVP-8-HARD controlled cohort, a 31-case hard-negative stress matrix, and selected model conditions. The repaired E0-E6 main result now covers Qwen, DeepSeek, and Gemini, but it is still not a broad-model result. The hard-negative stress matrix reaches three projects, but the scrapy cases are curated no-API stress-source variants, so it should not be generalized as a pure agent-generated realistic cohort. The results therefore support bounded evidence-conditioned risk behavior in a controlled patch-verifier study, not universal LLM patch-verifier reliability.

Baseline validity. Rule-only visible-tool is the completed deterministic baseline. Always-* and uniform-random policies are reference policies. Majority-vote and a separate deterministic E0/no-tool verifier remain unavailable under the current tracked-summary boundary.

VIII. CONCLUSION

This paper introduces EVP-8 as a hidden-evaluator evidence-visibility protocol for candidate patch verification. The repaired Qwen, DeepSeek, and Gemini v0.3 results show

TABLE V
HARD-NEGATIVE STRESS MATRIX AGGREGATE RESULTS.

condition	records	repeated false accepts	strict rejects	safe escalations	safe handling
Current prompt	93	62 (66.67%)	0 (0.00%)	31 (33.33%)	31 (33.33%)
Coverage prompt	93	12 (12.90%)	0 (0.00%)	81 (87.10%)	81 (87.10%)

TABLE VI
FALSE-ACCEPT ANATOMY AND REPRESENTATIVE CASE GROUPS.

project/task	negative type	models accepting	likely cause
thefuck / thefuck_1	Regression patch	DeepSeek, Gemini, Qwen	visible E6 evidence missed a hidden P2P-broad regression
youtube-dl / youtube-dl_11	Partial fix	DeepSeek, Gemini	visible tests encouraged acceptance despite partial semantic repair
youtube-dl / youtube-dl_16	Partial fix	DeepSeek, Gemini, Qwen	visible tests encouraged acceptance despite partial semantic repair
youtube-dl / youtube-dl_20	Partial fix	DeepSeek, Gemini, Qwen	visible tests encouraged acceptance despite partial semantic repair
youtube-dl / youtube-dl_6	Partial fix	Gemini, Qwen	merge-gate summary did not expose the remaining semantic gap

that visible executable and tool evidence can unlock correct-patch acceptance while retaining false-accept risk and model-dependent caution. E6 ablations, tool-contestation, coverage-contestation, and the hard-negative stress matrix further show that verdict-like evidence and prompt framing can shape policy behavior. EVP-8’s engineering value is to reveal when LLMs behave as tool followers, abstainers, or risk routers under explicit evidence boundaries. The contribution is a reproducible software-engineering protocol for measuring evidence-conditioned risk behavior in a controlled LLM patch-verifier study, not a claim of autonomous patch correctness verification.

REFERENCES

- [1] Z. Qi, F. Long, S. Achour, and M. Rinard, “An analysis of patch plausibility and correctness for generate-and-validate patch generation systems,” in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, 2015, pp. 24–36.
- [2] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “A systematic study of automated program repair: Fixing 55 out of 105 bugs for \$8 each,” in *Proceedings of the 34th International Conference on Software Engineering*, 2012, pp. 3–13.
- [3] C. K. Chow, “On optimum recognition error and reject tradeoff,” *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970.
- [4] C. Cortes, G. DeSalvo, and M. Mohri, “Learning with rejection,” *Journal of Machine Learning Research*, vol. 17, no. 63, pp. 1–40, 2016.
- [5] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4878–4887.
- [6] F. Long and M. Rinard, “Automatic patch generation by learning correct code,” in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2016, pp. 298–312.
- [7] —, “Staged program repair with condition synthesis,” in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 166–178.
- [8] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, “Semfix: Program repair via semantic analysis,” in *Proceedings of the 35th International Conference on Software Engineering*, 2013, pp. 772–781.
- [9] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, “Is the cure worse than the disease? overfitting in automated program repair,” in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543.
- [10] R. Just, D. Jalali, and M. D. Ernst, “Defects4j: A database of existing faults to enable controlled testing studies for java programs,” in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 437–440.
- [11] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, “Bugsinspy: A database of existing bugs in python programs to enable controlled testing and debugging studies,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1556–1560.
- [12] F. Madeiral, S. Urli, M. Maia, and M. Monperrus, “Bears: An extensible java bug benchmark for automatic program repair studies,” in *Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering*, 2019, pp. 468–478.
- [13] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, “Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge,” in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 2017, pp. 55–56.
- [14] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [15] C. S. Xia and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering*, 2023, pp. 1482–1494.
- [16] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, “An empirical investigation into learning bug-fixing patches in the wild via neural machine translation,” in *Proceedings of the 41st International Conference on Software Engineering*, 2019, pp. 832–837.
- [17] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [18] H. Joshi, J. Cambronero, S. Gulwani, V. Le, I. Radicek, and G. Verbruggen, “Repair is nearly generation: Multilingual program repair with llms,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5131–5140.
- [19] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proceedings of the 35th International Conference on Software Engineering*, 2013, pp. 712–721.
- [20] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu, and N. Sundaresan, “Automating code review activities by large-scale pre-training,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1035–1047.
- [21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world

github issues?” in *Proceedings of the 12th International Conference on Learning Representations*, 2024.

- [22] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, 2024.
- [23] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *Advances in Neural Information Processing Systems*, 2023.
- [24] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui, “Large language models are not fair evaluators,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 9440–9450.
- [25] R. Parasuraman and V. Riley, “Humans and automation: Use, misuse, disuse, abuse,” *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997.